# Parallel Conjugate Gradient for the 2D and 3D Poisson Equation with MPI, MPI+OpenMP and MPI+CUDA

DSA5208 Project 3 AY2026/27 Semester 1 National University of Singapore


| | | |
|---|---|---|
| Yikun Zhao | A0353894M | serial and MPI solvers, validation, report |
| Jiahao Zhou | A0354090J | MPI+OpenMP and MPI+CUDA solvers |
| Zheke Chen | A0356140M | cluster builds, experiments, analysis and figures |


13 November 2026


**Abstract**

We solve the Poisson equation $-\Delta u = f$ on the unit square and the unit cube with an unpreconditioned, matrix-free conjugate gradient (CG) method and parallelise it in three ways: pure MPI with a domain decomposition, hybrid MPI+OpenMP, and MPI+CUDA with one GPU per rank. All versions share one CG driver and are validated against a manufactured solution (observed second-order accuracy) and against the serial solver (identical iteration counts, pointwise agreement to rounding error). On the Vanda cluster, pure MPI reaches 138× speedup on 256 cores for a 3D problem with $2.4 \times 10^7$ unknowns, but the 2D problem stops scaling beyond one 64-core node because every CG iteration requires two global reductions whose latency over the inter-node network ($\approx$ 150–300 $\mu$s each) exceeds the remaining computation. Cartesian blocks beat 1D slabs by 5× in 3D, a single-reduction CG variant recovers up to 35% at 256 cores, hybrid MPI+OpenMP does not beat pure MPI at fixed core count, and one NVIDIA A40 GPU is about 19× faster than one CPU core.


## Contents


1 Introduction and the algorithm 2

2 Implementation and parallelisation design 3

3 Validation 5

4 Experimental setup 6

5 Serial runtime scaling 6

6 MPI performance 8

7 MPI+X performance 14

8 Overall discussion 17

9 Conclusion 18

# 1 Introduction and the algorithm

## 1.1 Model problem and discretisation

We solve

$$-\Delta u = f \quad \text{in } \Omega = (0,1)^d, \qquad u = 0 \quad \text{on } \partial\Omega, \qquad d \in \{2,3\}. \tag{1}$$

The domain is covered by a uniform grid with $n$ interior points per side, spacing $h = 1/(n+1)$ and points $x_i = (i+1)h$, $i = 0, \ldots, n-1$. The standard second-order finite-difference approximation of $-\Delta$ uses the 5-point stencil in 2D and the 7-point stencil in 3D,

$$(A\mathbf{u})_{\mathbf{i}} = \frac{1}{h^2}\Big(2d\,u_{\mathbf{i}} - \sum_{k=1}^{d}\big(u_{\mathbf{i}+\mathbf{e}_k} + u_{\mathbf{i}-\mathbf{e}_k}\big)\Big), \tag{2}$$

with boundary values set to zero. This gives a linear system $A\mathbf{u} = \mathbf{b}$ with $N = n^d$ unknowns. $A$ is sparse (at most $2d+1$ non-zeros per row, in total $\mathrm{nnz}(A) = (2d+1)n^d - 2d\,n^{d-1}$), symmetric and positive definite (SPD), with condition number $\kappa(A) = \mathcal{O}(h^{-2}) = \mathcal{O}(n^2)$.

## 1.2 The conjugate gradient method

CG [1] is the standard Krylov method for SPD systems. Starting from $\mathbf{x}_0 = 0$, each iteration performs one matrix–vector product, two inner products and three vector updates (Algorithm 1). We stop when the relative residual $\|\mathbf{r}_k\|_2/\|\mathbf{b}\|_2$ falls below a tolerance (the program default is $10^{-8}$; all timed experiments used $5 \times 10^{-9}$, see Section 4). Without a preconditioner the number of iterations grows like $\sqrt{\kappa} = \mathcal{O}(n)$; we deliberately keep CG unpreconditioned so that every iteration has exactly the same communication structure and the parallel overheads can be studied in isolation.

**Algorithm 1** Standard CG ("`std`"): two global reductions per iteration

$\mathbf{x} = 0,\ \mathbf{r} = \mathbf{b},\ \mathbf{p} = \mathbf{r},\ \rho = \mathbf{r}^\top\mathbf{r}$
**for** $k = 1, 2, \ldots$ **do**
  $\mathbf{q} = A\mathbf{p}$ ▷ stencil sweep; needs neighbour values (halo exchange)
  $\alpha = \rho/(\mathbf{p}^\top\mathbf{q})$ ▷ global reduction #1
  $\mathbf{x} \leftarrow \mathbf{x} + \alpha\mathbf{p}, \quad \mathbf{r} \leftarrow \mathbf{r} - \alpha\mathbf{q}$
  $\rho_{\text{new}} = \mathbf{r}^\top\mathbf{r}$ ▷ global reduction #2
  **if** $\sqrt{\rho_{\text{new}}}/\|\mathbf{b}\| < \text{tol}$ **then stop**
  **end if**
  $\mathbf{p} \leftarrow \mathbf{r} + (\rho_{\text{new}}/\rho)\,\mathbf{p}, \quad \rho \leftarrow \rho_{\text{new}}$
**end for**

We also implement the Chronopoulos–Gear variant [2] ("`pipe`" in our code), which computes $\gamma = \mathbf{r}^\top\mathbf{r}$ and $\delta = (A\mathbf{r})^\top\mathbf{r}$ together in *one* reduction per iteration and updates the extra vector $\mathbf{s} = A\mathbf{p}$ by recurrence ($\mathbf{s} \leftarrow \mathbf{w} + \beta\mathbf{s}$ with $\mathbf{w} = A\mathbf{r}$). It produces the same iterates in exact arithmetic, trades one global reduction for two extra vector updates, and is the standard remedy when reduction latency dominates.

## 1.3 Why this algorithm is not embarrassingly parallel

The matrix is never stored (a *matrix-free* method): $A\mathbf{p}$ is a stencil sweep over the grid. When the grid is distributed, a point next to a sub-domain boundary needs values owned by a neighbouring process, so every iteration requires a *halo exchange* (point-to-point communication). Each inner product needs contributions from all processes, so every iteration also requires two *global reductions* (`MPI_Allreduce`) that act as synchronisation points: no process can compute $\alpha$ or $\beta$ before all others have finished their part. The serial dependency between consecutive iterations means the iterations cannot be distributed; only the work *within* an iteration can.

### 1.4 Synthetic data

All data are synthetic and generated on the fly by every process for the points it owns. We use the manufactured solution

$$u(\mathbf{x}) = \prod_{k=1}^{d} g(x_k), \qquad g(t) = t(1-t)e^t, \qquad f = -\Delta u = \sum_{k=1}^{d} t_k(t_k+3)e^{t_k} \prod_{j\neq k} g(x_j)\Big|_{t=\mathbf{x}}, \tag{3}$$

which vanishes on $\partial\Omega$ and has non-zero fourth derivatives, so the discretisation error is exactly computable and of size $\mathcal{O}(h^2)$. The right-hand side is $\mathbf{b_i} = f(\mathbf{x_i})$. We deliberately did *not* use the common choice $u = \prod_k \sin(\pi x_k)$: its grid samples form an exact eigenvector of $A$, so CG converges in a single iteration and any timing would be meaningless. With our choice CG needs $\approx 3.2n$ iterations in 2D and $\approx 3.8n$ in 3D (Table 3), as for a generic right-hand side. Problem size is controlled only by $n$, which makes weak-scaling meshes easy to construct.

### 1.5 Terminology

| Term | Meaning in this report |
|---|---|
| rank | One MPI process. Each rank is pinned to its own physical core (or cores, with OpenMP). |
| halo / ghost layer | One extra layer of grid points around a rank's block holding copies of the neighbours' boundary values. |
| slab / Cartesian decomposition | Split of the grid into 1D slabs along the slowest axis, or into 2D/3D blocks over all axes (`MPI_Dims_create`/`MPI_Cart_create`). |
| blocking / overlapped halo | `MPI_Sendrecv` before the stencil, or non-blocking `MPI_Isend/Irecv` with the interior stencil computed while messages are in flight. |
| `std` / `pipe` CG | Standard CG (2 reductions per iteration) / Chronopoulos–Gear single-reduction CG (1 reduction). |
| strong scaling | Fixed global problem, increasing core count $P$. Speedup $S(P) = T(1)/T(P)$, efficiency $E(P) = S(P)/P$, where $T(1)$ is the one-rank MPI run. |
| weak scaling | Fixed work per core ($N/P$ constant). Weak efficiency $E_w(P) = T(1)/T(P)$; per-iteration weak efficiency uses time per CG iteration. |
| superlinear speedup | $S(P) > P$, here caused by the working set fitting in cache once it is split over enough cores. |
| memory-bandwidth bound | Runtime limited by the rate at which data is moved from main memory, not by arithmetic. |
| true residual | $\|\mathbf{b} - A\mathbf{x}\|/\|\mathbf{b}\|$ recomputed after the solve, as opposed to the recursively updated $\|\mathbf{r}_k\|$. |
| CUDA-aware MPI (device transport) | MPI accepts GPU memory pointers directly. In the *staged* transport the halo is copied to host memory first. |
| `MPI_THREAD_FUNNELED` | Thread-support level in which only the master thread makes MPI calls. |
| phase times | Per-rank accumulated time in the *stencil* (stencil and vector updates), *halo* (neighbour exchange) and *reduce* (local dot product, `MPI_Allreduce` and any wait for slower ranks) phases; we report the maximum over ranks. |

## 2 Implementation and parallelisation design

### 2.1 Software structure

The CG algorithm (both variants) is written once, in `src/common/cg.cpp`, against an abstract `Backend` interface: vector allocation, `apply` ($\mathbf{q} = A\mathbf{p}$, including any halo exchange), global `dot` and fused `dot2`, and the vector updates `axpy`/`xpay`. The serial, MPI and MPI+OpenMP solvers are three implementations of this interface; MPI+OpenMP inherits the MPI backend and replaces only its compute loops. The CUDA solver keeps its own CG loop on the device but uses the same problem definition, command-line interface and output format. Consequently all CPU versions

execute the identical sequence of floating-point operations up to the order of summation, which is what makes the strict serial-versus-parallel validation in Section 3 possible. No BLAS, LAPACK or other linear-algebra library is used. Table 1 summarises how each part of CG is parallelised in each version.

**Table 1:** How each part of one CG iteration is parallelised.

| Operation | MPI | MPI+OpenMP | MPI+CUDA |
|---|---|---|---|
| $A\mathbf{p}$ (stencil) | owned block per rank; halo by subarray datatypes; blocking or overlapped | + rows of the block split over threads (`collapse(2)`, inner $x$ loop vectorised) | one GPU thread per point; faces packed by a kernel, sent device-direct or staged through host |
| dot products | local sum + `MPI_Allreduce` (2 per iteration, or 1 fused) | + OpenMP `reduction` before the `Allreduce` | block-level shared-memory tree, second kernel, copy of one scalar to host, `Allreduce` |
| vector updates | local, no communication | `parallel for simd` | grid-stride kernels |

## 2.2 Data layout

Each rank stores its owned block of $L_z \times L_y \times L_x$ points plus one ghost layer in a contiguous row-major array ($x$ fastest). Ghosts on the physical boundary hold the Dirichlet value 0 and are never written, so the stencil loop has no boundary branches and the inner $x$ loop vectorises. In 2D the $z$ axis is inactive and carries no ghost layer.

## 2.3 MPI design

**Domain decomposition.** With `-decomp slab` the slowest axis ($z$ in 3D, $y$ in 2D) is split into $P$ slabs; each rank has at most two neighbours and every halo face is contiguous. With `-decomp cart`, `MPI_Dims_create` factorises $P$ into a balanced 2D or 3D process grid and `MPI_Cart_create` provides the neighbours. When $P$ does not divide $n$, the first $n \bmod P$ coordinates receive one extra point, so loads differ by at most one plane. Table 2 shows the consequence for communication volume: per iteration a Cartesian 3D block of the $n = 288$ mesh on 256 ranks sends 101 KiB, against 1.3 MiB for a slab; and at 288 ranks a slab is a single plane thick, so it sends twice as many values as it owns.

**Table 2:** Halo traffic per rank and per CG iteration (largest block) for the strong-scaling meshes. "ratio" is the number of values sent divided by the number of points owned (surface-to-volume).

| | | slab | | | | Cartesian | | | |
|---|---|---|---|---|---|---|---|---|---|
| Mesh | $P$ | grid | nbrs | KiB | ratio | grid | nbrs | KiB | ratio |
| 2D, $n = 2048$ | 64 | $64 \times 1$ | 2 | 32.0 | 0.062 | $8 \times 8$ | 4 | 8.0 | 0.016 |
| 2D, $n = 2048$ | 256 | $256 \times 1$ | 2 | 32.0 | 0.250 | $16 \times 16$ | 4 | 4.0 | 0.031 |
| 2D, $n = 2048$ | 288 | $288 \times 1$ | 2 | 32.0 | 0.250 | $24 \times 12$ | 4 | 4.0 | 0.035 |
| 3D, $n = 288$ | 64 | $64 \times 1 \times 1$ | 2 | 1296.0 | 0.400 | $4 \times 4 \times 4$ | 6 | 243.0 | 0.083 |
| 3D, $n = 288$ | 256 | $256 \times 1 \times 1$ | 2 | 1296.0 | 1.000 | $8 \times 8 \times 4$ | 6 | 101.2 | 0.139 |
| 3D, $n = 288$ | 288 | $288 \times 1 \times 1$ | 2 | 1296.0 | 2.000 | $8 \times 6 \times 6$ | 6 | 90.0 | 0.139 |

**Halo exchange.** Faces are described by `MPI_Type_create_subarray` on the extended array, so the same code serves both decompositions and both dimensions without manual packing. Only face neighbours are exchanged because the stencil has no diagonal terms. In `-halo block` mode each decomposed axis performs two `MPI_Sendrecv` calls. In `-halo overlap` mode all receives and sends are posted with `MPI_Irecv/Isend`, the stencil is applied to the interior points that need no ghost

value, `MPI_Waitall` completes the exchange, and finally the boundary shell (decomposed into at most six disjoint boxes) is computed.

**Global reductions.** Each rank sums its owned points and one `MPI_Allreduce` combines the partial sums; `pipe` CG packs both scalars into one two-element `Allreduce`. Because every rank needs the result before it can continue, reductions are synchronisation points; any imbalance or operating-system noise appears as waiting time in the reduce phase.

### 2.4 MPI+OpenMP design

The hybrid solver keeps the MPI decomposition, halo exchange and reductions and threads the compute kernels inside each rank. MPI is initialised with `MPI_THREAD_FUNNELED`; all MPI calls happen on the master thread outside parallel regions. The stencil distributes $(z, y)$ rows over threads with `collapse(2)` and static scheduling, keeping the contiguous $x$ loop inside one thread so that it remains vectorised; dot products use an OpenMP `reduction` clause, and vector updates use `parallel for simd`. At a fixed number of cores, using $T$ threads per rank divides the number of MPI ranks by $T$: fewer ranks take part in each `Allreduce` and the total halo surface shrinks, at the cost of fork–join overheads, threads idling during communication, and non-uniform memory access (NUMA) when a rank's threads span both sockets of a node. Threads are pinned with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`.

### 2.5 MPI+CUDA design

Each rank drives one GPU. All vectors stay in device memory for the whole solve. A grid-stride kernel applies the stencil; dot products use a shared-memory tree reduction per thread block, a second kernel to sum the block partials, a copy of one scalar to the host and an `MPI_Allreduce`. For the halo, a kernel packs each strided face into a contiguous device buffer; the buffer is then either passed directly to MPI (*device*, CUDA-aware MPI) or copied to host memory first (*staged*), and the received face is unpacked by another kernel. The CUDA solver implements standard CG with blocking halo exchange; timers synchronise the device before each phase ends.

## 3 Validation

Correctness is established by a small set of deterministic tests (run by `ctest`; seven test groups) and by checks applied to every timed run.

1. **Operator.** For random vectors, $|(A\mathbf{x})^\top \mathbf{y} - \mathbf{x}^\top A\mathbf{y}|/|(A\mathbf{x})^\top \mathbf{y}| \approx 2 \times 10^{-15}$ and $\mathbf{x}^\top A\mathbf{x} > 0$: the implemented operator is symmetric and positive definite, as CG requires.
2. **Discretisation order.** The discrete $L^2$ error $e = \left(h^d \sum_{\mathbf{i}} (x_{\mathbf{i}} - u(\mathbf{x}_{\mathbf{i}}))^2\right)^{1/2}$ must decrease as $h^2$. Table 3 and Figure 1 (bottom) show an observed order of 2.00 at every refinement in both 2D and 3D, from $n = 128$ to 2048 (2D) and $n = 32$ to 288 (3D).
3. **CG variants.** `std` and `pipe` CG give the same iteration count and the same error to $10^{-6}$ relative.
4. **Parallel equals serial.** For every combination of decomposition (slab, Cartesian), halo (blocking, overlapped) and CG variant, in 2D and 3D, on 1, 3 and 4 MPI ranks (and with 2 OpenMP threads per rank for the hybrid solver), the parallel solution is compared point by point with the serial solution at the same global index. The grids include sizes that do not divide evenly ($n = 37$, $n = 13$) and blocks only one point thick, which exercises the overlapped-halo code path with an empty interior. All cases give the same iteration count as the serial solver and a maximum pointwise difference of $\approx 10^{-15}$ relative to $\max|u|$, i.e. rounding error from the different summation order. The CUDA kernels are checked separately against a CPU stencil, dot product and face packing on irregular test data with non-zero ghost values (`src/hybrid_cuda/checks/kernel_check.cu`).

5. **Every timed run.** Each of the 339 accepted timed runs (279 CPU, 60 GPU, including the small-mesh preflight runs) reports the true residual, which was below $10^{-8}$ in all cases (maximum $5.6 \times 10^{-9}$), and was required to match a serial run on the same mesh in iteration count (within one) and error (to $10^{-6}$ relative). In practice the iteration counts were identical across all implementations, decompositions and core counts (e.g. 6512 for 2D $n = 2048$, 1109 for 3D $n = 288$), so speedups compare identical amounts of work.

# 4 Experimental setup

All runs were performed on the Vanda cluster. CPU runs used four exclusive nodes, each with two Intel Xeon Platinum 8452Y processors (36 cores per socket, 72 cores per node, one hardware thread per core), i.e. up to 288 physical cores; every rank (or thread) was pinned to its own physical core and placements were verified to be disjoint. Ranks were packed onto as few nodes as possible: up to 64 ranks on one node, 128 on two, and 256 or 288 on four. GPU runs used NVIDIA A40 GPUs, one per rank and one per node; the queue limited a job to two GPUs. The code was compiled in Release mode (`-O3`, C++17) with NVIDIA HPC SDK 26.1 and its HPC-X Open MPI 4.1.9. Inter-node messages used Open MPI's TCP transport (the hardware-accelerated collective libraries failed to initialise and were disabled), so absolute communication costs are those of TCP, not of RDMA.

**Measurement protocol.** The reported time covers the CG iteration loop only, between two barriers, excluding allocation, initialisation, error computation and output. It is the maximum over ranks. Each configuration was run as one discarded warm-up plus three independent processes; we report the minimum of the three (the least perturbed run) and quote the median where the spread is large. All timed runs stop at a recursive relative residual of $5 \times 10^{-9}$ (at most 20000 iterations), which leaves a margin for the acceptance test on the recomputed true residual ($< 10^{-8}$). Phase times come from the same run. Because phases are maxima over ranks that may come from different ranks, and because the reduce phase also absorbs waiting time, the three phase times need not add up to the total. All times, sizes and core counts are plotted on logarithmic axes.

# 5 Serial runtime scaling

Table 3 and Figure 1 vary the problem size: the matrix dimension $N$ grows from $1.6 \times 10^4$ to $4.2 \times 10^6$ (2D) and from $3.3 \times 10^4$ to $2.4 \times 10^7$ (3D), and the number of non-zeros from $8 \times 10^4$ to $1.7 \times 10^8$. Going from 2D to 3D also changes the non-zeros per row from 5 to 7. The cost of one CG iteration is linear in $\mathrm{nnz}(A)$, and the number of iterations grows like $n$, so the expected total cost is $\mathcal{O}(N^{3/2})$ in 2D and $\mathcal{O}(N^{4/3})$ in 3D. The fitted slopes of Figure 1 are 1.62 (2D) and 1.43 (3D), slightly above these predictions, and the column "ns/(it·nnz)" explains why: the time per iteration and non-zero is $\approx 0.45$ ns while the working set fits in cache and rises to 0.65–0.86 ns once it no longer does. One iteration of standard CG touches about 14 vectors of length $N$ (stencil input and output, four dot-product operands, three updates of three operands each), i.e. $\approx 112$ bytes per unknown. For 2D $n = 2048$ this is 470 MB per iteration in 18 ms, $\approx 26$ GB/s, which is the single-core memory bandwidth: serial CG is memory-bandwidth bound.

**Table 3:** Serial runtime scaling (minimum of three runs). $N = n^d$ unknowns; nnz non-zeros of $A$; order $= \log(e_{\text{prev}}/e)/\log(h_{\text{prev}}/h)$.

| dim | $n$ | $N$ | nnz | iters | time (s) | ms/it | ns/(it·nnz) | $L^2$ error | order |
|---|---|---|---|---|---|---|---|---|---|
| 2D | 128 | 16,384 | 81,408 | 392 | 0.015 | 0.038 | 0.46 | $2.92 \times 10^{-6}$ | |
| 2D | 256 | 65,536 | 326,656 | 789 | 0.132 | 0.167 | 0.51 | $7.36 \times 10^{-7}$ | 2.00 |
| 2D | 512 | 262,144 | 1,308,672 | 1594 | 1.716 | 1.077 | 0.82 | $1.85 \times 10^{-7}$ | 2.00 |
| 2D | 1024 | 1,048,576 | 5,238,784 | 3224 | 10.634 | 3.298 | 0.63 | $4.63 \times 10^{-8}$ | 2.00 |
| 2D | 2048 | 4,194,304 | 20,963,328 | 6512 | 117.3 | 18.006 | 0.86 | $1.16 \times 10^{-8}$ | 2.00 |
| 3D | 32 | 32,768 | 223,232 | 119 | 0.011 | 0.095 | 0.43 | $1.38 \times 10^{-5}$ | |
| 3D | 64 | 262,144 | 1,810,432 | 241 | 0.329 | 1.365 | 0.75 | $3.57 \times 10^{-6}$ | 2.00 |
| 3D | 128 | 2,097,152 | 14,581,760 | 485 | 3.392 | 6.995 | 0.48 | $9.05 \times 10^{-7}$ | 2.00 |
| 3D | 192 | 7,077,888 | 49,324,032 | 734 | 23.585 | 32.132 | 0.65 | $4.04 \times 10^{-7}$ | 2.00 |
| 3D | 203 | 8,365,427 | 58,310,735 | 775 | 29.978 | 38.681 | 0.66 | $3.62 \times 10^{-7}$ | 2.00 |
| 3D | 288 | 23,887,872 | 166,717,440 | 1109 | 144.6 | 130.420 | 0.78 | $1.80 \times 10^{-7}$ | 2.00 |

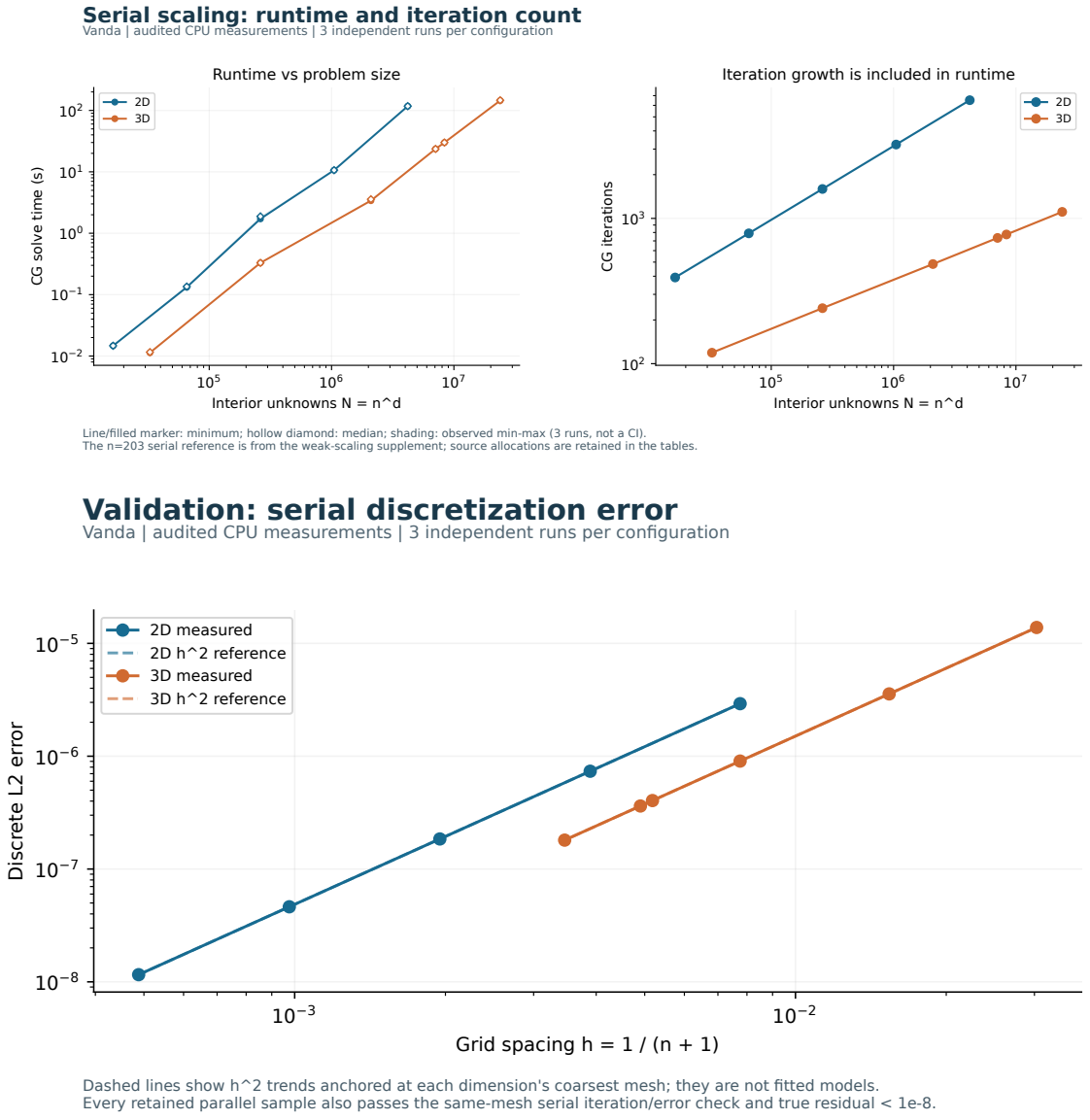



**Figure 1:** Top: serial runtime and CG iteration count against the number of unknowns. Bottom: discretisation error against $h$ with $h^2$ reference lines (validation of Section 3).

# 6 MPI performance

## 6.1 Strong scaling

We use the largest meshes that fit our time budget: 2D $n = 2048$ ($4.2 \times 10^6$ unknowns) and 3D $n = 288$ ($2.4 \times 10^7$ unknowns), from 1 to 288 cores (Tables 4–5, Figures 2–3).

**Table 4:** 2D strong scaling, $n = 2048$, standard CG, blocking halo. $S$ speedup, $E$ efficiency (%); phase times are for the Cartesian runs.

| | | Cartesian | | | slab | | | Cartesian phases (s) | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $P$ | nodes | $T$ (s) | $S$ | $E$ | $T$ (s) | $S$ | $E$ | stencil | halo | reduce |
| 1 | 1 | 117.183 | 1.0 | 100.0 | 117.679 | 1.0 | 100.0 | 86.796 | 0.002 | 30.379 |
| 2 | 1 | 62.673 | 1.9 | 93.5 | 63.004 | 1.9 | 93.4 | 46.111 | 0.624 | 16.355 |
| 4 | 1 | 32.425 | 3.6 | 90.4 | 32.063 | 3.7 | 91.8 | 23.842 | 0.732 | 8.926 |
| 8 | 1 | 18.542 | 6.3 | 79.0 | 18.274 | 6.4 | 80.5 | 13.777 | 0.395 | 5.355 |
| 16 | 1 | 10.091 | 11.6 | 72.6 | 10.176 | 11.6 | 72.3 | 7.629 | 0.334 | 2.501 |
| 32 | 1 | 4.581 | 25.6 | 79.9 | 4.711 | 25.0 | 78.1 | 3.670 | 0.253 | 1.047 |
| 64 | 1 | 1.266 | 92.5 | 144.6 | 1.385 | 85.0 | 132.8 | 0.827 | 0.126 | 0.532 |
| 128 | 2 | 2.814 | 41.6 | 32.5 | 2.746 | 42.8 | 33.5 | 0.408 | 0.973 | 2.430 |
| 256 | 4 | 2.194 | 53.4 | 20.9 | 1.966 | 59.9 | 23.4 | 0.213 | 0.728 | 1.967 |
| 288 | 4 | 2.235 | 52.4 | 18.2 | 2.259 | 52.1 | 18.1 | 0.193 | 0.531 | 2.052 |

**Table 5:** 3D strong scaling, $n = 288$, standard CG, blocking halo.

| | | Cartesian | | | slab | | | Cartesian phases (s) | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $P$ | nodes | $T$ (s) | $S$ | $E$ | $T$ (s) | $S$ | $E$ | stencil | halo | reduce |
| 1 | 1 | 146.632 | 1.0 | 100.0 | 146.551 | 1.0 | 100.0 | 112.857 | 0.000 | 33.773 |
| 2 | 1 | 78.139 | 1.9 | 93.8 | 78.027 | 1.9 | 93.9 | 59.688 | 0.509 | 19.247 |
| 4 | 1 | 36.103 | 4.1 | 101.5 | 40.720 | 3.6 | 90.0 | 26.378 | 0.599 | 10.281 |
| 8 | 1 | 24.137 | 6.1 | 75.9 | 24.810 | 5.9 | 73.8 | 17.508 | 1.060 | 5.998 |
| 16 | 1 | 17.243 | 8.5 | 53.1 | 18.043 | 8.1 | 50.8 | 13.061 | 0.704 | 4.057 |
| 32 | 1 | 14.354 | 10.2 | 31.9 | 16.203 | 9.0 | 28.3 | 11.090 | 0.504 | 3.350 |
| 64 | 1 | 8.020 | 18.3 | 28.6 | 10.895 | 13.5 | 21.0 | 5.964 | 0.773 | 2.918 |
| 128 | 2 | 3.349 | 43.8 | 34.2 | 7.336 | 20.0 | 15.6 | 2.354 | 0.524 | 1.578 |
| 256 | 4 | 1.065 | 137.7 | 53.8 | 5.465 | 26.8 | 10.5 | 0.405 | 0.442 | 0.642 |
| 288 | 4 | 1.061 | 138.2 | 48.0 | 4.732 | 31.0 | 10.8 | 0.335 | 0.439 | 0.696 |

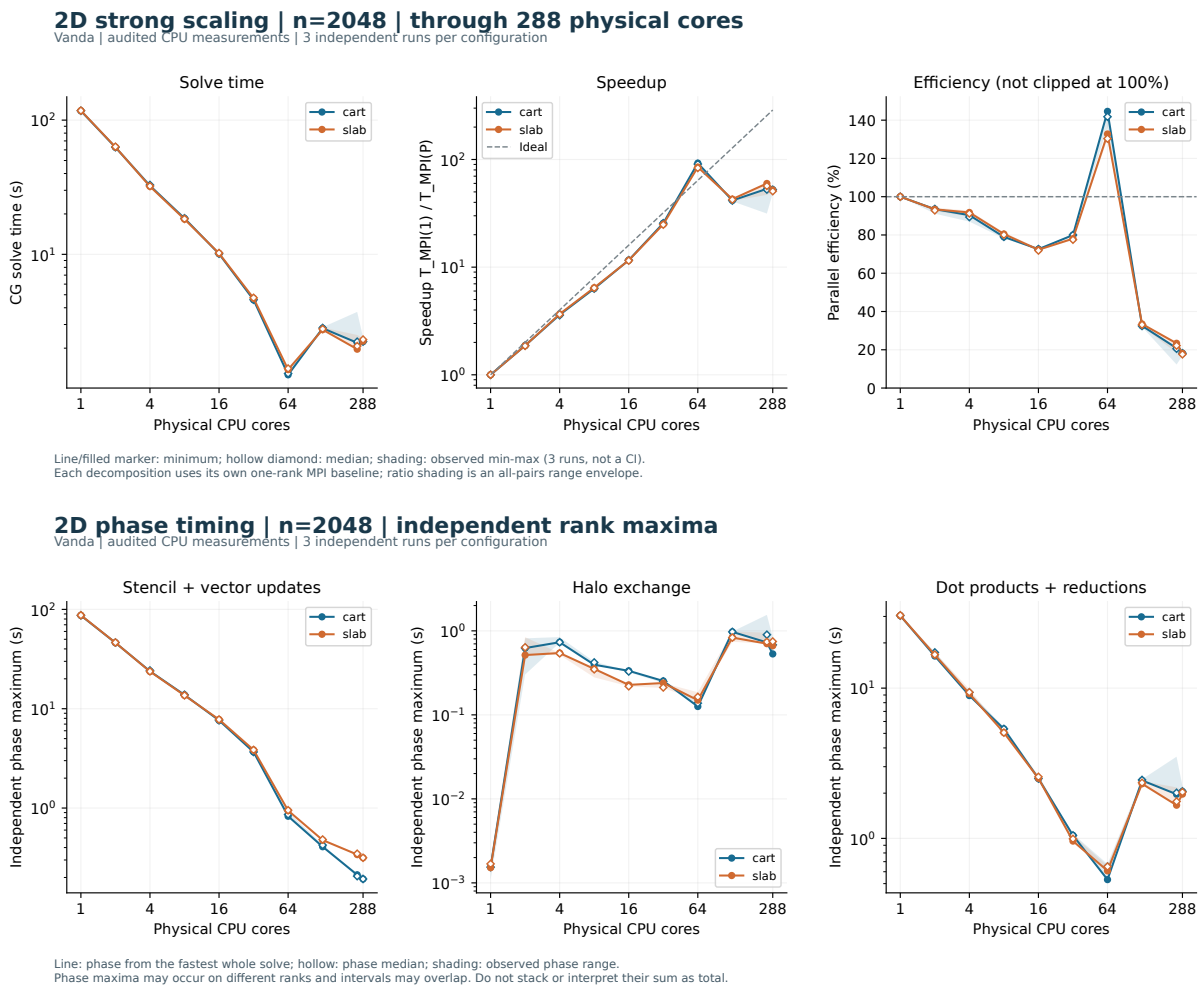



**Figure 2:** 2D strong scaling ($n = 2048$): time, speedup and efficiency (top) and per-phase maxima (bottom). Lines: minimum of three runs; hollow diamonds: median; shading: observed range.

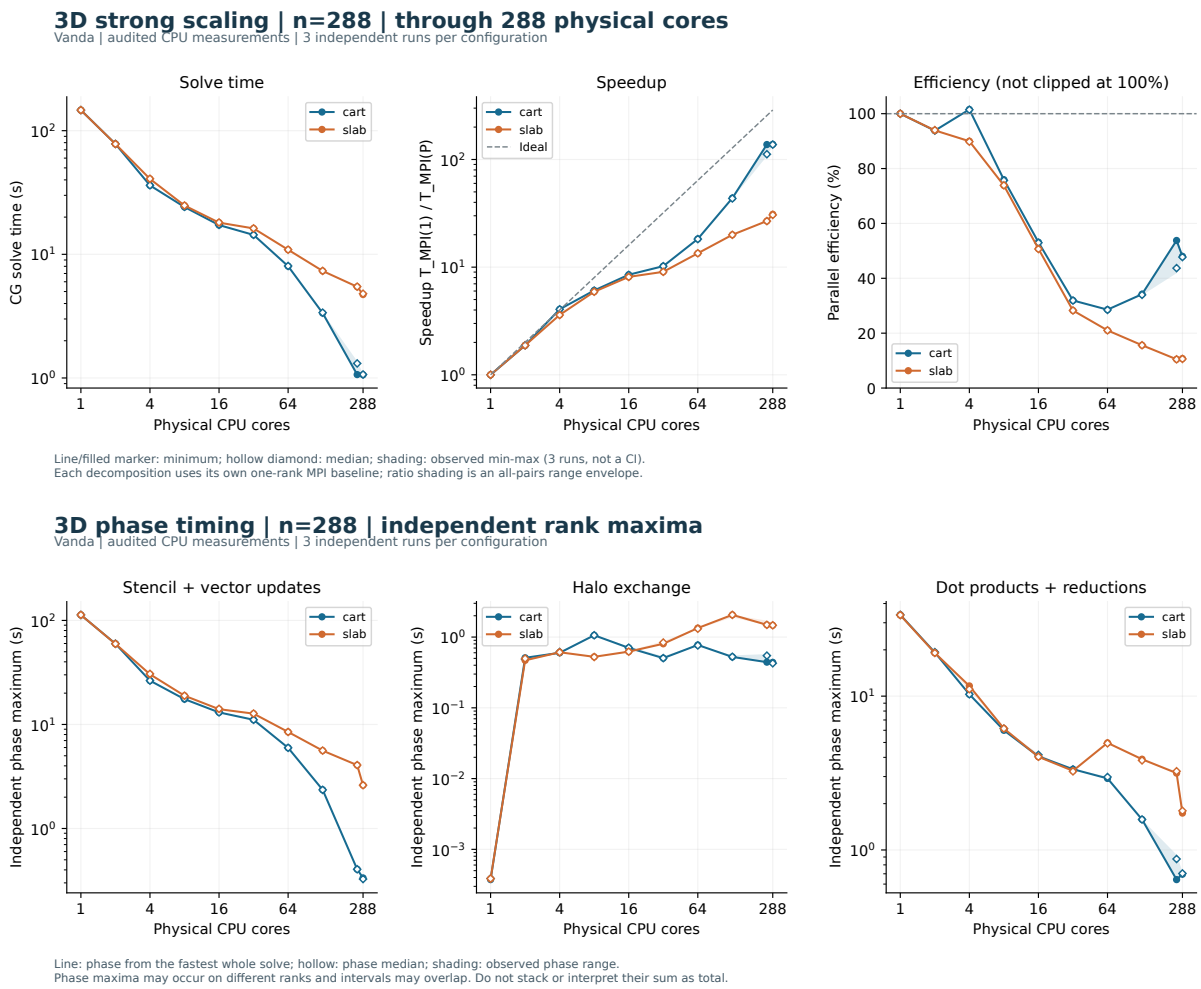



**Figure 3:** 3D strong scaling ($n = 288$) and per-phase maxima.

**Within one node (1–64 cores) the solver is limited by memory, not communication.** Halo times stay below 1 s for all $P$, a small fraction of the total, and in 2D the efficiency stays at 72–93% up to 32 cores. The two meshes then behave very differently at 64 cores. The 2D working set (five vectors of $4.2 \times 10^6$ doubles, $\approx 170$ MB) fits in the aggregate L2 and L3 caches of one node, so the 64-rank 2D run moves $\approx 470$ MB per iteration in 0.19 ms, a rate only caches can supply, and becomes *superlinear* ($S = 92.5$, $E = 145\%$). The 3D working set ($\approx 1$ GB) does not fit: at 64 cores the estimated traffic is $\approx 400$ GB/s, which is close to the sustainable memory bandwidth of one two-socket node, so adding cores within the node stops helping and the 3D efficiency falls to 29%. When the 3D problem is spread over two and then four nodes, both the aggregate memory bandwidth and the aggregate cache grow, and the stencil time falls from 5.96 s (64 cores) to 0.41 s (256 cores), a 14.7× reduction for 4× the cores. This is why the 3D curve recovers to $S = 138$ at 256 cores ($E = 54\%$).

**Across nodes the global reductions dominate.** In 2D the time *increases* from 1.27 s on one node to 2.81 s on two nodes, and stays at $\approx 2.2$ s on four. The phase data show why: the reduce time jumps from 0.53 s to 2.43 s as soon as the job spans two nodes. Dividing the reduce time by the $2 \times 6512$ reductions gives the average cost of one reduction (Table 6): 41 $\mu$s inside a node (mostly the local dot product) but 150–190 $\mu$s once the `Allreduce` crosses the TCP network. The 2D problem at 256 cores has only 16,384 points per rank, so the computation per iteration (about 33 $\mu$s of stencil work) is far smaller than one inter-node reduction; the algorithm is *latency bound* and adding cores cannot help. The 3D problem has 5.7× more points and 6× fewer iterations, so at

256 cores its stencil work per iteration ($\approx 370$ $\mu$s) still exceeds the reduction latency and the solver keeps scaling. Within one node the reduce phase also contains waiting time: the slowest rank of each iteration determines when the `Allreduce` completes.

**Table 6:** Average cost of one reduction (reduce phase divided by 2×iterations), Cartesian, standard CG. It includes the local dot product, the `MPI_Allreduce` and waiting for the slowest rank.

| dim | $P$ | nodes | $\mu$s per reduction |
|---|---|---|---|
| 2D | 1 | 1 | 2332.6 |
| 2D | 16 | 1 | 192.0 |
| 2D | 64 | 1 | 40.8 |
| 2D | 128 | 2 | 186.6 |
| 2D | 256 | 4 | 151.0 |
| 2D | 288 | 4 | 157.6 |
| 3D | 1 | 1 | 15226.8 |
| 3D | 16 | 1 | 1829.0 |
| 3D | 64 | 1 | 1315.7 |
| 3D | 128 | 2 | 711.5 |
| 3D | 256 | 4 | 289.3 |
| 3D | 288 | 4 | 313.8 |

**Decomposition.** In 2D slab and Cartesian decompositions perform alike (their halo volumes of 32 KiB and 4–8 KiB per iteration are both negligible beside the reductions). In 3D the Cartesian decomposition is 5.1× faster at 256 cores (1.06 s against 5.46 s): a 3D slab face is a full $288 \times 288$ plane, and with 256 or 288 slabs each rank owns only one or two planes, so it sends as many values as it owns (Table 2) and its stencil is applied to blocks that are too thin to vectorise or reuse cache well. Cartesian decomposition is the only choice that scales in 3D.

## 6.2 Weak scaling

For weak scaling the mesh grows with $P$ so that the number of unknowns per core is constant: $n = 128\sqrt{P}$ in 2D (16,384 points per core) and $n = \mathrm{round}(32\sqrt[3]{P})$ in 3D (32,768 per core; $-0.3\%$ at $P = 256$ due to rounding), up to 256 cores (Table 7, Figure 4).

**Table 7:** Weak scaling (Cartesian, standard CG). $E_w = T(1)/T(P)$ for the whole solve; $E_{\mathrm{it}}$ uses the time per iteration.

| dim | $P$ | $n$ | $N/P$ | iters | $T$ min (s) | $T$ median (s) | $\mu$s/it | $E_w$ (%) | $E_{\mathrm{it}}$ (%) |
|---|---|---|---|---|---|---|---|---|---|
| 2D | 1 | 128 | 16,384 | 392 | 0.0092 | 0.0093 | 23.6 | 100.0 | 100.0 |
| 2D | 4 | 256 | 16,384 | 789 | 0.0277 | 0.0278 | 35.1 | 33.3 | 67.1 |
| 2D | 16 | 512 | 16,384 | 1594 | 0.0845 | 0.0858 | 53.0 | 10.9 | 44.5 |
| 2D | 64 | 1024 | 16,384 | 3224 | 0.1902 | 0.1933 | 59.0 | 4.9 | 40.0 |
| 2D | 256 | 2048 | 16,384 | 6512 | 2.1263 | 4.0904 | 326.5 | 0.4 | 7.2 |
| 3D | 1 | 32 | 32,768 | 119 | 0.0073 | 0.0074 | 60.9 | 100.0 | 100.0 |
| 3D | 8 | 64 | 32,768 | 241 | 0.0377 | 0.0379 | 156.6 | 19.2 | 38.9 |
| 3D | 64 | 128 | 32,768 | 485 | 0.0933 | 0.0938 | 192.4 | 7.8 | 31.7 |
| 3D | 216 | 192 | 32,768 | 734 | 0.4916 | 0.5123 | 669.7 | 1.5 | 9.1 |
| 3D | 256 | 203 | 32,677 | 775 | 0.4646 | 0.5308 | 599.4 | 1.6 | 10.2 |

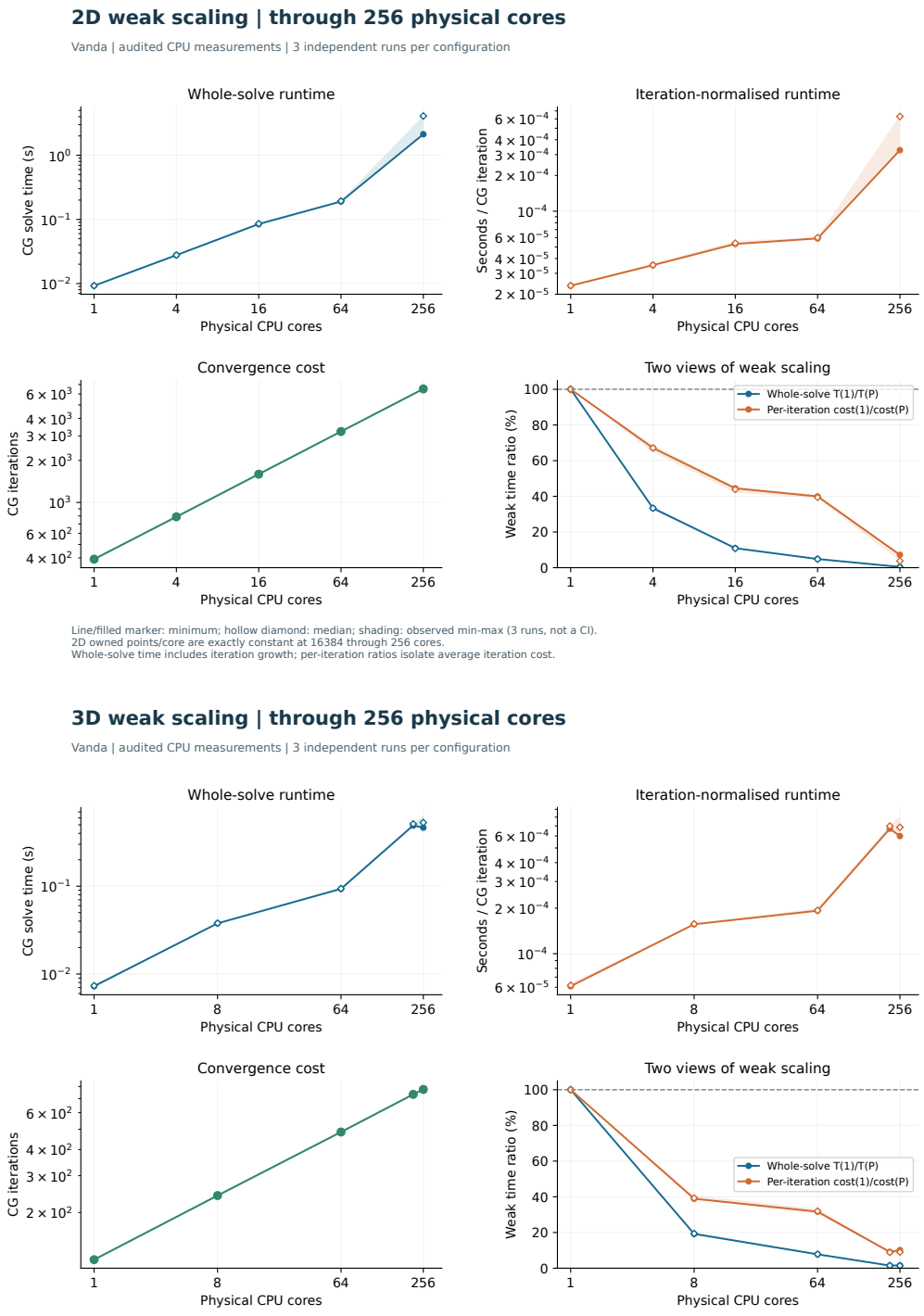



**Figure 4:** Weak scaling in 2D (top) and 3D (bottom): whole-solve and per-iteration time, weak efficiency, and iteration growth.

Whole-solve weak efficiency necessarily decays for unpreconditioned CG, because the iteration count grows with $n$ (392 to 6512 in 2D, 119 to 775 in 3D) even though the work per iteration per core is constant. The per-iteration efficiency $E_{\text{it}}$ removes this algorithmic effect and isolates the parallel overhead. It falls to 40% (2D) and 32% (3D) at 64 cores on one node, where 64 ranks share the memory bandwidth that a single rank had alone, and then to 7–10% at 216–256 cores on several nodes, where each iteration pays two inter-node reductions: at 2D $P = 256$, 291 $\mu$s of the 327 $\mu$s per iteration is reduce time. The 2D endpoint is also the noisiest point of the study (median 1.9× the

minimum), as expected for a run dominated by network latency.

### 6.3 Communication-reducing variants

Table 8 and Figure 5 compare the four combinations of halo mode and CG variant at 64 cores (one node) and 256 cores (four nodes).

**Table 8:** Solve time (s) for the four halo/CG choices (Cartesian; best per row in bold), and reduce phase of the blocking runs.

| | | total time (s) | | | | reduce phase (s) | |
|---|---|---|---|---|---|---|---|
| dim | $P$ | block/std | overlap/std | block/pipe | overlap/pipe | std | pipe |
| 2D | 64 | **1.266** | 1.282 | 1.743 | 1.756 | 0.532 | 0.521 |
| 2D | 256 | 2.194 | 2.010 | **1.425** | 1.504 | 1.967 | 1.159 |
| 3D | 64 | **8.020** | 8.200 | 8.801 | 8.985 | 2.918 | 1.579 |
| 3D | 256 | 1.065 | **0.855** | 1.263 | 1.200 | 0.642 | 0.423 |

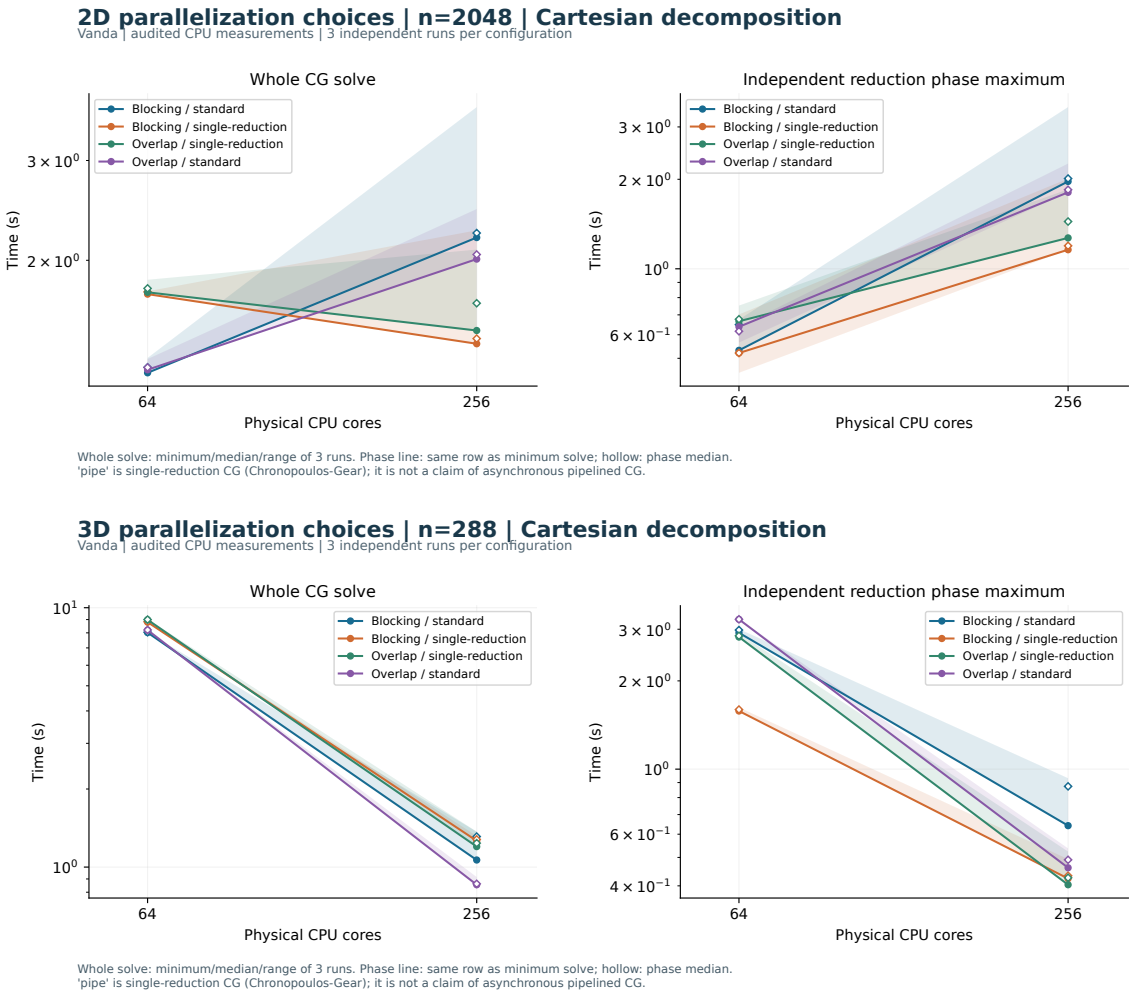



**Figure 5:** Halo mode and CG variant at 64 and 256 cores, 2D (top) and 3D (bottom).

The single-reduction CG addresses exactly the bottleneck identified above. In the latency-bound 2D case at 256 cores it reduces the reduce phase from 1.97 s to 1.16 s and the total time by 35% (2.19 s to 1.42 s). Where the solver is still bandwidth bound (64 cores in both dimensions, and 3D at 256 cores) its two extra vector updates cost more than the saved reduction, and it is 6–38%

slower. Overlapping the halo with interior computation helps when the halo is significant and there is enough interior work to hide it: 3D at 256 cores runs 20% faster (0.85 s against 1.06 s; halo phase 0.18 s against 0.44 s). It does not help in 2D, where the halo is small. There is therefore no universally best variant: the right choice depends on whether bandwidth, neighbour messages or global latency dominates, which is precisely what the phase breakdown measures.

# 7 MPI+X performance

## 7.1 MPI+OpenMP at a fixed number of cores

We use all 288 cores of four nodes and vary the split between MPI ranks and OpenMP threads per rank, from $288 \times 1$ to $4 \times 72$ (one rank per socket would be $8 \times 36$; $4 \times 72$ is one rank per node). Table 9 and Figure 6 give the results relative to pure MPI with 288 ranks.

**Table 9:** Fixed 288 cores: MPI ranks $\times$ OpenMP threads. Ratio $= T/T_{\text{pure MPI}}$ (larger is slower).

| | | | 2D, $n = 2048$ | | 3D, $n = 288$ | |
|---|---|---|---|---|---|---|
| version | ranks | threads | $T$ (s) | ratio | $T$ (s) | ratio |
| pure MPI | 288 | 1 | 2.235 | 1.00 | 1.061 | 1.00 |
| MPI+OpenMP | 288 | 1 | 2.290 | 1.02 | 1.078 | 1.02 |
| MPI+OpenMP | 144 | 2 | 3.527 | 1.58 | 1.304 | 1.23 |
| MPI+OpenMP | 72 | 4 | 3.516 | 1.57 | 1.379 | 1.30 |
| MPI+OpenMP | 32 | 9 | 2.635 | 1.18 | 1.764 | 1.66 |
| MPI+OpenMP | 16 | 18 | 2.504 | 1.12 | 2.153 | 2.03 |
| MPI+OpenMP | 4 | 72 | 2.678 | 1.20 | 2.653 | 2.50 |

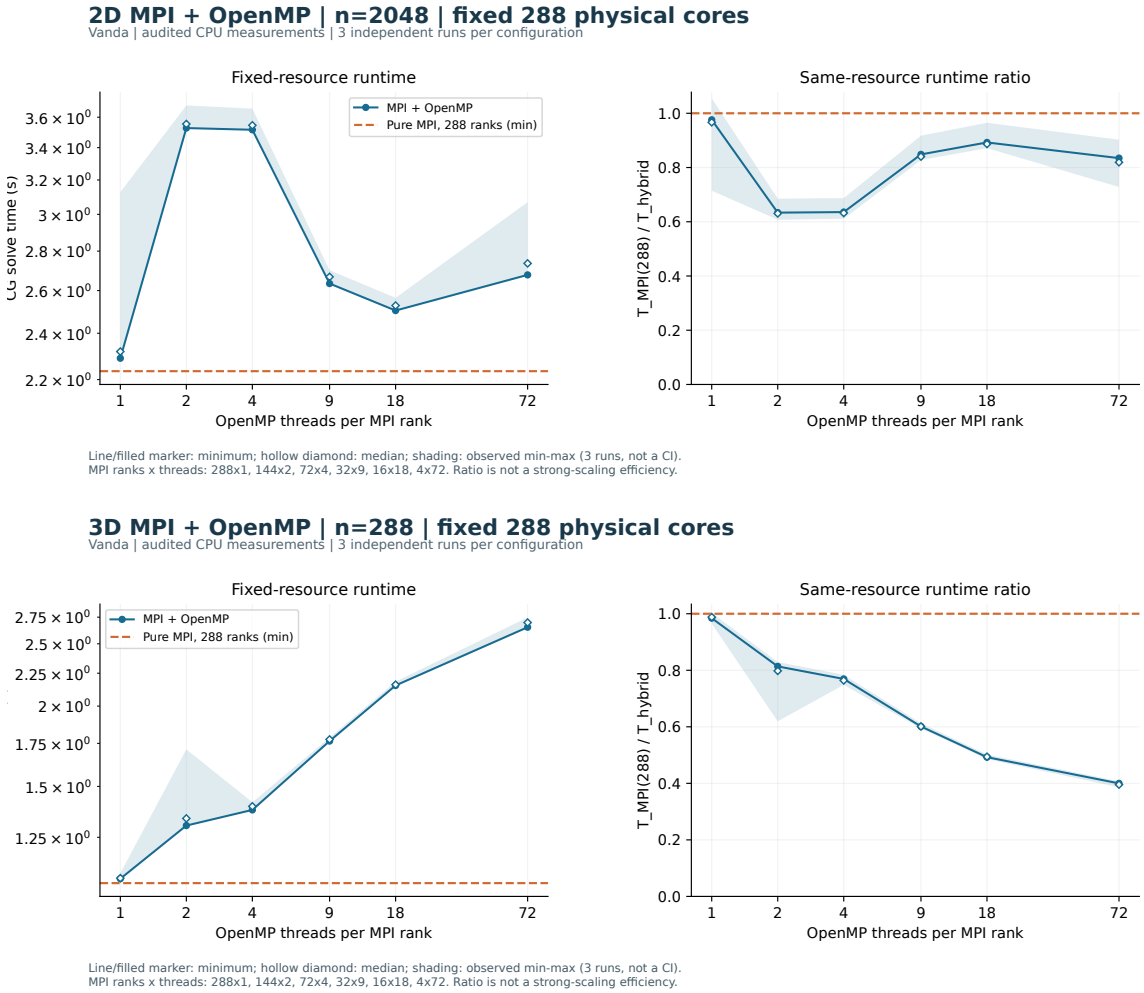



**Figure 6:** MPI ranks × OpenMP threads on 288 cores, 2D (top) and 3D (bottom).

**Table 10:** Phase maxima (s) of the hybrid runs.

| dim | ranks×threads | stencil | halo | reduce |
|---|---|---|---|---|
| 2D | 288×1 | 0.199 | 0.652 | 2.092 |
| 2D | 144×2 | 0.274 | 0.963 | 3.190 |
| 2D | 72×4 | 0.228 | 0.995 | 3.109 |
| 2D | 32×9 | 0.267 | 0.912 | 2.056 |
| 2D | 16×18 | 0.287 | 1.014 | 1.671 |
| 2D | 4×72 | 0.664 | 1.135 | 1.151 |
| 3D | 288×1 | 0.337 | 0.431 | 0.704 |
| 3D | 144×2 | 0.345 | 0.593 | 0.815 |
| 3D | 72×4 | 0.302 | 0.655 | 0.713 |
| 3D | 32×9 | 0.307 | 1.143 | 0.512 |
| 3D | 16×18 | 0.370 | 1.490 | 0.575 |
| 3D | 4×72 | 0.729 | 1.667 | 0.533 |

With one thread per rank the hybrid solver matches pure MPI to within 2%, so the threading layer itself costs little. Adding threads did not make the solver faster for these meshes; the best multi-threaded splits were $16 \times 18$ in 2D (12% slower than pure MPI) and $144 \times 2$ in 3D (23% slower). The phase breakdown (Table 10) shows three competing effects.

- *Fewer ranks in each reduction.* In 2D the reduce phase drops from 2.09 s ($288 \times 1$) to 1.15 s ($4 \times 72$), confirming that hybrid parallelism lowers global-reduction cost. With $144 \times 2$ and $72 \times 4$,

however, it rises to $\approx 3.1$ s; these splits leave ranks of a node spread over both sockets with small thread teams, and the extra fork–join and imbalance appears as waiting time in the reductions.

- *Funnelled communication.* Halo time grows monotonically with the thread count (3D: 0.43 s to 1.67 s). Fewer, larger blocks have larger faces, and with `MPI_THREAD_FUNNELED` the master thread alone packs and sends them while the other threads wait.
- *NUMA.* With 72 threads per rank the stencil time more than doubles (2D: 0.20 s to 0.66 s). Vectors are zero-initialised by the master thread, so by the first-touch page placement policy all of a rank's memory resides on one socket and half of its threads read remote memory. Parallel first-touch initialisation would remove this effect.

In 3D, where reductions matter less, the loss of halo efficiency dominates and every multi-threaded split is slower. In 2D, the reduction saving and the halo/NUMA costs nearly cancel.

### 7.2 MPI+CUDA on GPUs

Table 11 and Figure 7 show the CUDA solver on one and two NVIDIA A40 GPUs (one per node), with both halo transports. The 3D GPU mesh is $n = 256$ ($1.7 \times 10^7$ unknowns), the largest whose serial reference fits the time budget; the last column compares with the serial CPU solver on the same mesh.

**Table 11:** MPI+CUDA strong scaling on A40 GPUs. $S$ and $E$ (%) are relative to one GPU with the same transport; "vs serial" is the speedup over one CPU core on the same mesh.

| mesh | transport | GPUs | $T$ (s) | $S$ | $E$ | stencil | halo | reduce | vs serial |
|---|---|---|---|---|---|---|---|---|---|
| 2D, $n = 2048$ | device | 1 | 6.215 | 1.00 | 100.0 | 4.629 | 0.000 | 1.585 | 18.5 |
| 2D, $n = 2048$ | staged | 1 | 6.214 | 1.00 | 100.0 | 4.628 | 0.000 | 1.585 | 18.5 |
| 2D, $n = 2048$ | device | 2 | 5.069 | 1.23 | 61.3 | 2.300 | 1.519 | 1.691 | 22.6 |
| 2D, $n = 2048$ | staged | 2 | 4.359 | 1.43 | 71.3 | 2.300 | 0.822 | 1.407 | 26.3 |
| 3D, $n = 256$ | device | 1 | 4.239 | 1.00 | 100.0 | 3.467 | 0.000 | 0.771 | 18.9 |
| 3D, $n = 256$ | staged | 1 | 4.238 | 1.00 | 100.0 | 3.467 | 0.000 | 0.771 | 18.9 |
| 3D, $n = 256$ | device | 2 | 2.984 | 1.42 | 71.0 | 1.664 | 0.857 | 0.486 | 26.8 |
| 3D, $n = 256$ | staged | 2 | 2.860 | 1.48 | 74.1 | 1.666 | 0.732 | 0.482 | 28.0 |

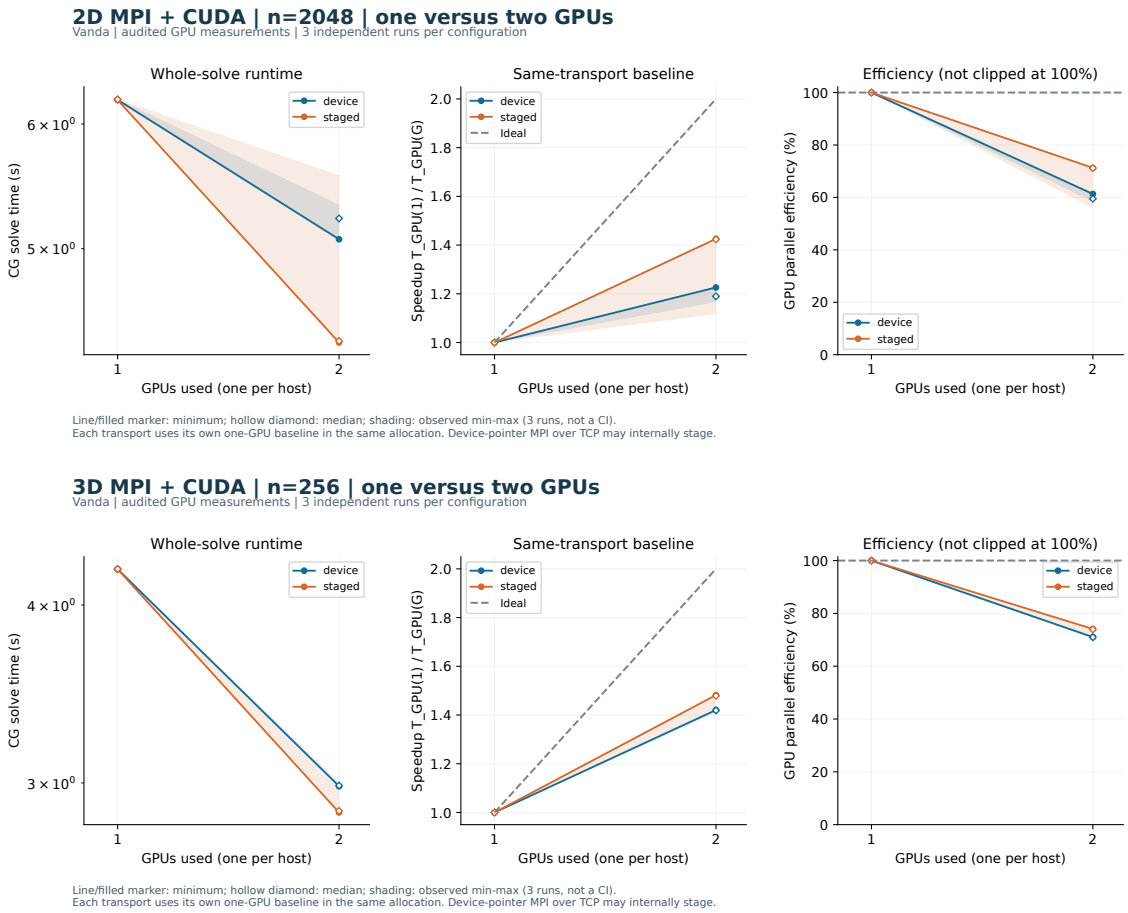



**Figure 7:** MPI+CUDA on one and two A40 GPUs, 2D (top) and 3D (bottom), staged and device transports.

One A40 solves the problems 18.5× (2D) and 18.9× (3D) faster than one CPU core, roughly the performance of 16–32 cores of one node. As on the CPU, the GPU solver is memory-bandwidth bound: with ≈ 112 bytes per unknown per iteration, the single-GPU runs sustain about 490 GB/s (2D) and 440 GB/s (3D), 60–70% of the A40's 696 GB/s peak; the A40's low double-precision arithmetic rate is therefore not the limit. The reduce phase is substantial even on one GPU (1.59 s in 2D, i.e. 120 $\mu$s per dot product): each reduction launches two kernels and copies a scalar back to the host, which synchronises the device. A fused single-reduction CG would help the GPU at least as much as the CPU.

With two GPUs on two nodes the stencil time halves exactly (2D: 4.63 s to 2.30 s), but each iteration now pays a halo exchange between nodes (0.73–1.52 s) and inter-node reductions, so the speedup is 1.23–1.48 (efficiency 61–74%). The staged transport was faster than device-pointer transport in both meshes. Since inter-node traffic used TCP, CUDA-aware MPI cannot transfer directly from GPU memory to the network and must stage the data internally, with its own synchronisation; explicit staging in our code is at least as efficient. The site limited jobs to two GPUs, so we could not measure beyond two devices.

# 8 Overall discussion

**How the problem was parallelised.** Parallelism comes from the data: the grid is split into blocks, and each operation of an iteration is parallelised with the cheapest technique for its data dependencies (Table 1). Vector updates are purely local. The stencil needs only nearest-neighbour data, handled by halo exchange between MPI ranks and by shared memory between threads and GPU threads. The dot products need data from every process and are the only global operation.

The design decisions that mattered most in our measurements were: (i) Cartesian rather than slab decomposition in 3D (5×); (ii) reducing the number of global reductions when they dominate (35% at 256 cores in 2D); (iii) overlapping halo exchange with interior work when halos are large (20% in 3D at 256 cores); and (iv) using one MPI rank per core rather than wide OpenMP teams, unless reduction latency is the main cost.

**Which parts need the most communication.** By volume, the halo exchange: tens to hundreds of KiB per rank per iteration, against 8–16 bytes for a reduction. By cost, the global reductions: they move almost no data, but they are latency bound, grow with $\log P$ and the number of nodes, and force all ranks to synchronise twice per iteration. The halo is a surface term that shrinks as $P$ grows (as $(N/P)^{(d-1)/d}$ for Cartesian blocks), whereas the reduction cost does not shrink at all. This is why 3D, with more work per point of surface and fewer iterations, scales to 256 cores while 2D stops at one node; and why the communication-avoiding variants that matter are those that reduce the *number* of global synchronisations rather than the message volume.

**Memory hierarchy.** Both CPU and GPU versions are memory-bandwidth bound. This explains the superlinear 2D speedup at 64 cores (the working set fits in cache), the efficiency loss when a single node fills up, the recovery of the 3D solver across nodes, and the $\approx 19\times$ GPU advantage, which is close to the ratio of GPU to single-core memory bandwidth.

**Limitations.** Inter-node communication used TCP, not RDMA, which makes reduction latency higher than on a tuned InfiniBand stack; the conclusions about where the cross-over from bandwidth-bound to latency-bound happens depend on this. Each configuration was run three times, so variability (up to 27% coefficient of variation at the 2D 256-core points) limits fine comparisons. GPU runs were limited to two devices. We did not use a preconditioner; a multigrid or block-Jacobi preconditioner would reduce the iteration count and therefore the number of reductions, at the cost of more communication per iteration.

# 9 Conclusion

We implemented a matrix-free CG solver for the 2D and 3D Poisson equation in serial, MPI, MPI+OpenMP and MPI+CUDA versions that share one algorithm and agree with each other to rounding error, and we validated second-order accuracy against a manufactured solution. Serial CG is memory-bandwidth bound and its cost grows like $N^{3/2}$ (2D) and $N^{4/3}$ (3D), slightly faster in practice once the working set leaves the cache. Pure MPI scales to 256 cores with 138× speedup for the 3D problem with Cartesian decomposition, while the 2D problem becomes bound by global-reduction latency once it spans more than one node; the single-reduction CG variant recovers 35% of that loss. At a fixed core count, hybrid MPI+OpenMP reduces the cost of the global reductions but did not outperform pure MPI because of funnelled communication and NUMA effects. One A40 GPU replaces about 16–32 CPU cores, and two GPUs give a 1.4–1.5× speedup limited by inter-node halo and reduction latency.

**Use of AI.** Generative AI tools (Anthropic Claude and OpenAI Codex) were used to assist in drafting code, tests, job scripts and this report; all output was reviewed, tested and validated by the authors, who take full responsibility for the content.

**Code.** The source code, build instructions and the scripts that reproduce every table and figure are at `https://github.com/ZYKuni/NUS_DSA5208_project3_1107` (see `README.md`).

# References


[1] M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.

[2] A. T. Chronopoulos and C. W. Gear. *s*-step iterative methods for symmetric linear systems. *Journal of Computational and Applied Mathematics*, 25(2):153–168, 1989.

[3] Y. Saad. *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, 2003.

[4] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023.

[5] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*, 2021.